



PROJECT DOCUMENTATION

Task Management System — TaskFlow

Subject: Full Stack Web Development

Project No.: 120

Program: B.Tech CSE (2025–29)

University: ITM Skills University

Semester: Semester III

Submitted By: Yuvraj Mishra

Roll Number: 150096725036

Cohort: Jeff Bezos

Submission: *GitHub Repository*

1. Project Overview

TaskFlow is a full-stack, role-based Task Management System built for a small company. Managers create tasks and assign them to employees with a due date, and employees view and update the status of the tasks assigned to them. An employee cannot modify a task that belongs to someone else, and tasks can be filtered by status and due date.

The system is made of two separate applications: a Node.js, Express.js and MongoDB backend that exposes a REST API and enforces every business and security rule, and a React frontend that only calls that API. Authentication uses JSON Web Tokens (JWT), and password recovery uses a security-question flow instead of email, so no SMTP service is required.

1.1 Problem Statement

A small company wants a backend where managers assign tasks to employees, and employees update the status of tasks assigned to them. An employee should not be able to modify a task that is not assigned to them, and tasks should be filterable by status and due date.

1.2 Objectives

- Design Employee, Manager and Task schemas using Mongoose.
- Implement role-based authorization for task assignment.
- Implement ownership-based authorization for status updates.
- Implement filtering by status and due date using query parameters.
- Validate task data on creation and update.
- Provide a responsive React frontend with task board, assignment form, status update and details views.

1.3 Expected Outcomes

- Managers can assign tasks with a due date to specific employees.
- Employees can update only the status of tasks assigned to them.
- Tasks can be filtered by status and due date.
- Unauthorized task modifications are rejected by the backend.

2. Technologies and Tools

TaskFlow uses the MERN-style stack described below. The backend and frontend live in separate folders of one project (backend/ and frontend/).

Layer	Technology	Purpose
Runtime	Node.js	Executes the backend server
Web framework	Express.js	REST API routing, middleware and controllers
Database	MongoDB with Mongoose	Stores employees, managers and tasks using referenced collections
Authentication	JSON Web Tokens (JWT)	Stateless login; token carries user id and role
Password security	bcryptjs	Hashes passwords and security answers (salt rounds: 10)
Configuration	dotenv (.env)	Keeps JWT_SECRET, MONGO_URI and PORT out of source code
Frontend	React with Vite	Single-page application and fast build tooling
Routing	React Router	Page navigation and protected routes
HTTP client	Axios	Calls the backend API and attaches the JWT
UI	Lucide React, Plus Jakarta Sans	Icons and the application typeface

2.1 Functional Modules

Module	Purpose	Main Function
Authentication	Register and sign in users	JWT login, role detection
Password Recovery	Reset a forgotten password without email	Security question, 3-step flow
Dashboard	Overview of workload and progress	Summary counts, calendar, charts
Task List	Browse and filter tasks	Status tabs, search, due-date filter, sort
Create Task	Let managers assign work	Title, description, employee, due date
Status Update	Let employees report progress	Pending / In Progress / Completed
Profile	Show account and permissions	Account details, activity, security info

3. System Architecture

TaskFlow follows a client–server architecture. The React frontend runs in the browser and communicates with the Express API over HTTP using JSON. The API runs on port 5001 and all endpoints are served under the /api prefix.

3.1 Request Flow

- The user signs in and the backend returns a JWT containing the user's id and role.

- The frontend stores the session in AuthContext and localStorage and sends the token on every request as an Authorization: Bearer header through a shared Axios service.
- The protect middleware verifies the token and loads the user on the server.
- Role middleware (authorize) and ownership middleware (checkTaskOwnership) decide whether the action is allowed.
- The controller validates the data, runs the Mongoose query and returns a JSON response with a suitable HTTP status code.

3.2 Project Structure

Area	Folders / Files	Responsibility
Backend	models, routes, controllers, middleware, validators	Data schemas, endpoints, business rules, authorization and input validation
Frontend	components, context, pages, services, utils	Reusable UI, authentication state, screens, API client and helpers

3.3 Database Design

MongoDB stores three main collections. Tasks reference the employee they are assigned to and the manager who created them.

Collection	Key Fields	Notes
Employee	name, email, password, securityQuestion, securityAnswer	Password and answer are stored only as bcrypt hashes
Manager	name, email, password, securityQuestion, securityAnswer	Separate model from Employee; same recovery fields
Task	title, description, status, dueDate, assignedTo	assignedTo references an Employee; a Manager reference records the creator
Recovery fields	resetAnswerAttempts, resetAnswerLockedUntil, resetPasswordToken, resetPasswordExpires	Used by the account lockout and single-use reset token logic

4. User Roles and Permissions

There are two roles, employee and manager, each backed by its own Mongoose model. The role is stored in the JWT payload. The frontend uses it to decide which controls to display, but the backend remains the real security boundary.

Capability	Employee	Manager
Log in and view profile	Yes	Yes
View tasks	Only tasks assigned to them	All tasks, with filters
Create a task	No	Yes
Assign a task to an employee	No	Yes
Edit task details (title, description, due date)	No	Yes
Update task status	Only their own assigned tasks	Yes
Delete a task	Only if assigned to them AND status is Completed	Any task
View employee list for assignment	No	Yes

Task statuses. The backend supports exactly three values: Pending, In Progress and Completed. The interface shows them as colour-coded badges.

Status	Badge Colour	Meaning
Pending	Amber	Task assigned but not started
In Progress	Blue	Employee is working on the task
Completed	Green	Work is finished

5. Landing Page

The landing page introduces TaskFlow to new visitors. The top navigation links to Features, How it works, Roles, Security and FAQ, with Login and Get Started buttons on the right. The hero section presents the headline “Effortless task management, for every team” with a short description of the manager and employee workflow.

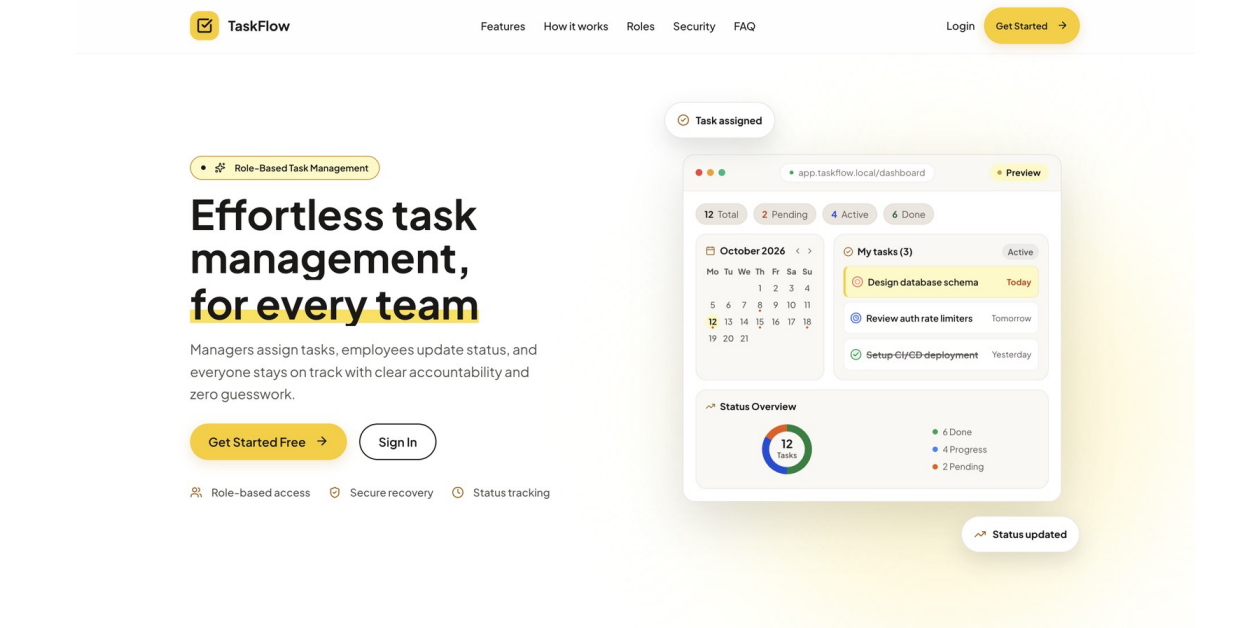


Figure 5.1 — TaskFlow landing page with hero section and dashboard preview.

The call-to-action buttons, Get Started Free and Sign In, lead to registration and login. Three highlights summarise the product: role-based access, secure recovery and status tracking. A dashboard preview on the right shows the calendar, a short task list and a status overview chart.

6. Authentication

6.1 Login

The login screen uses a split layout. The left panel carries the TaskFlow branding and tagline “Assign. Track. Complete.”, and the right panel holds the sign-in form.

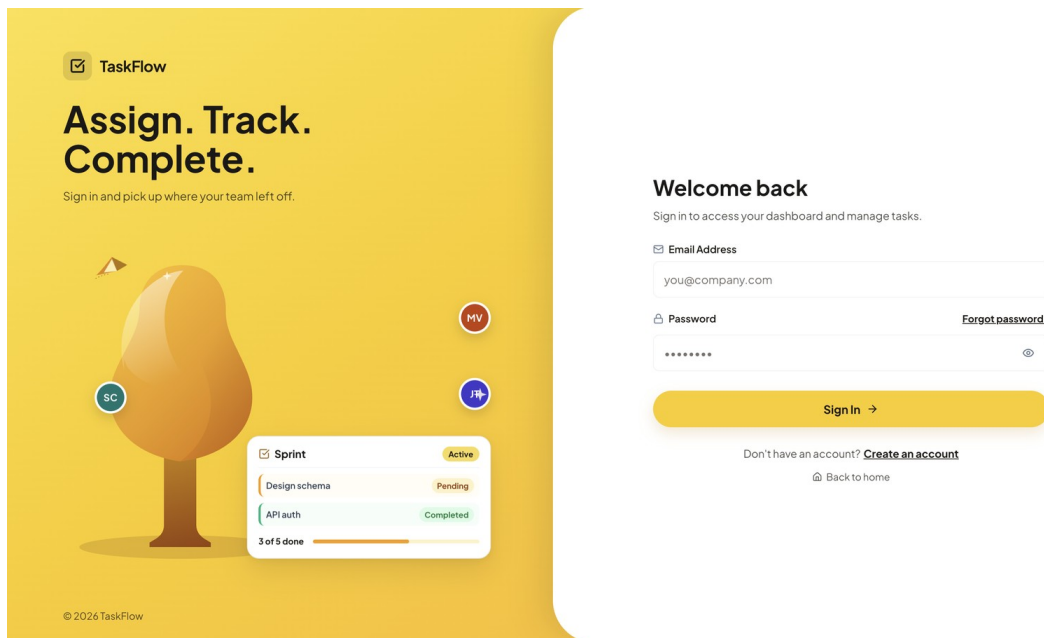


Figure 6.1 — Login screen with email and password fields.

The form has an Email Address field, a Password field with a show/hide toggle, a Forgot password? link, the Sign In button, a Create an account link and a Back to home link. While the request is running the button shows a loading state, and API errors such as invalid credentials are displayed as clear messages.

6.2 Registration

The registration form collects the user’s name, email, password, a security question and the security answer. A short note explains that the question is used to recover the account if the password is forgotten. The available security questions are:

- What is your school name?
- What is your favorite book?
- What is your favorite teacher’s name?
- What is your childhood nickname?

The backend validates the required fields, hashes both the password and the security answer with bcrypt before saving, and creates the account.

6.3 Session Handling with JWT

- On successful login the backend issues a JWT that is valid for 30 days and contains the user id and role.

- AuthContext keeps the signed-in user in React state and in localStorage so the session survives a page refresh.
- The shared Axios service (services/api.js) automatically adds the Authorization: Bearer header.
- The navigation bar and page controls change according to the role in the session.

7. Password Recovery

Password recovery does not need email or SMTP. It uses the security question that the user registered. The flow has three steps, shown to the user as a progress indicator: Account, Security and New Password.

Step	Screen	Endpoint	What Happens
1	Account	POST /api/auth/forgot-password	User enters their email. The response is generic whether or not the email exists, which prevents account enumeration. If the account exists, its security question is returned.
2	Security	POST /api/auth/verify-security-answer	User answers the question. The answer is compared with the stored hash. A correct answer returns a reset token.
3	New Password	POST /api/auth/reset-password	User enters and confirms a new password. The token is validated, the password is re-hashed and the token is invalidated.

After a successful reset the page shows “Password reset successfully” with a Back to Login button. The user is not logged in automatically and must sign in with the new password.

7.1 Protection Mechanisms

Mechanism	Implementation
Rate limiting	The three recovery endpoints allow 5 requests per 15 minutes.
Account lockout	After 5 consecutive wrong answers, resetAnswerLockedUntil is set to 15 minutes ahead. Further attempts return HTTP 429 with a lockout message until it expires.
Attempt tracking	resetAnswerAttempts counts failures and is reset after a successful verification or password reset.
Reset token	A 32-byte random token is generated with Node crypto. Only its SHA-256 hash is stored, and it expires after 15 minutes.
Single use	After the password is updated, the stored token and expiry are cleared so the token cannot be reused.
Hashed answers	Security answers are stored as bcrypt hashes, never as plain text.

8. Dashboard

The dashboard is the main control centre after login. A left sidebar provides navigation, and the top bar contains a search box, the New task button for managers, and the signed-in user's name with a role badge. The greeting is built from the authenticated user's name rather than hard-coded text.

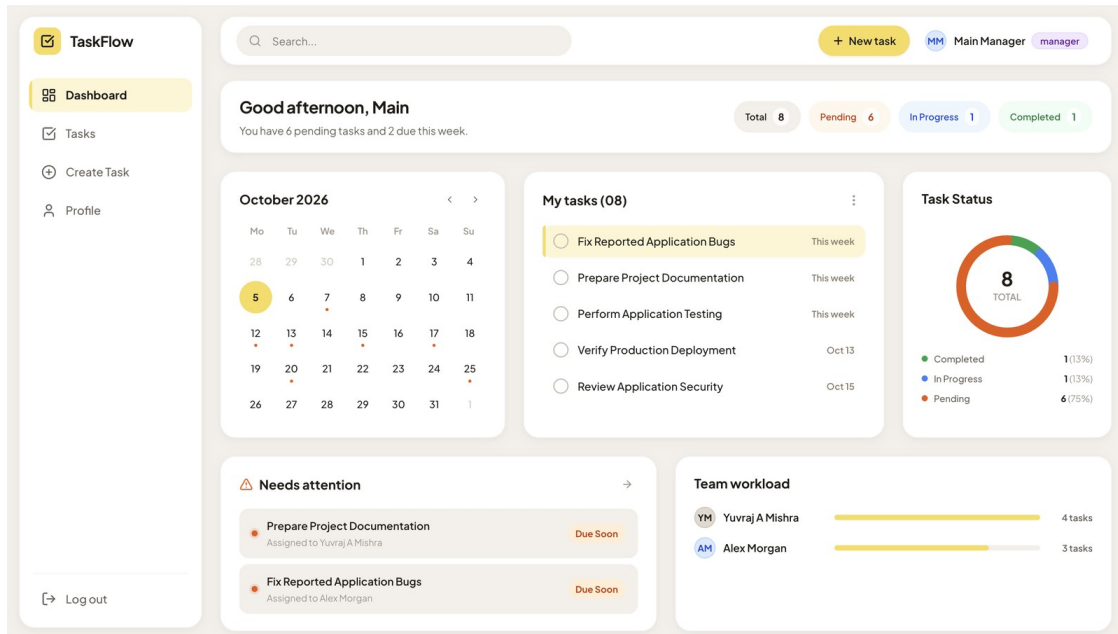


Figure 8.1 — Manager dashboard with summary, calendar, task list and charts.

8.1 Dashboard Components

- **Greeting and summary chips:** shows a personalised greeting, a line such as “You have 6 pending tasks and 2 due this week”, and the counts for Total, Pending, In Progress and Completed tasks.
- **Calendar:** a monthly calendar that highlights today and marks days that have tasks due.
- **My tasks:** a short list of upcoming tasks with a due label such as “This week” or a date.
- **Task Status chart:** a donut chart with the share of Completed, In Progress and Pending tasks.
- **Needs attention:** tasks that are due soon, with the assigned employee.
- **Team workload:** a bar view of how many tasks each employee currently has (shown to managers).

If there are no tasks, the dashboard shows an empty state (“No tasks yet. Your tasks will appear here.”).

9. Task Management

The Tasks page is the most important screen. It lists tasks as cards and provides every filtering and sorting control in one toolbar.

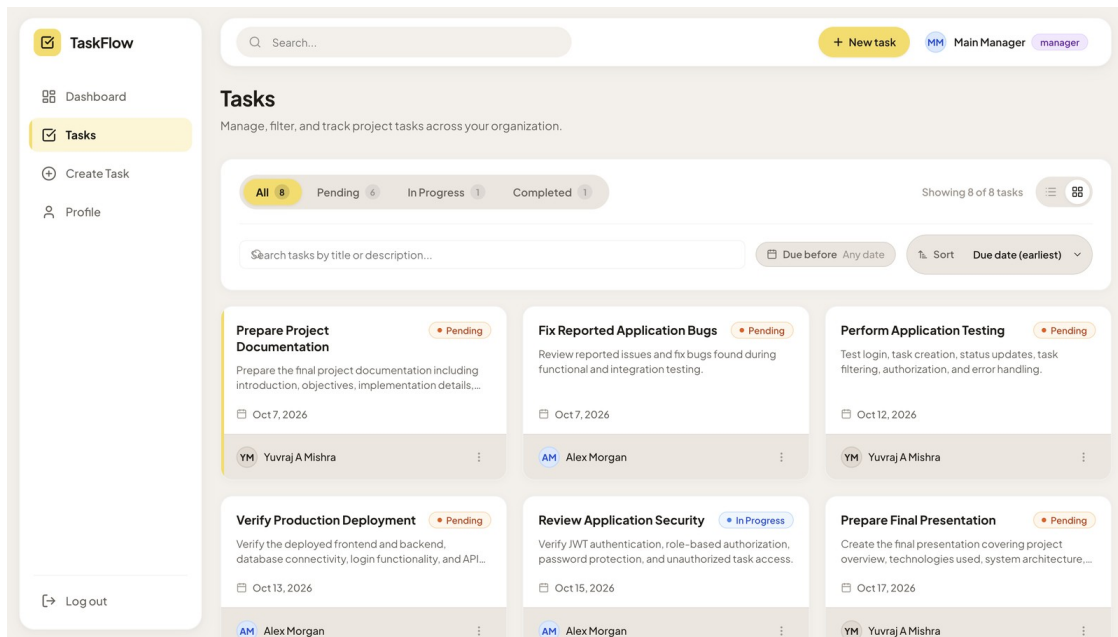


Figure 9.1 — Tasks page in grid view with status tabs, search, due-date filter and sort.

9.1 Task Cards

Each card shows the task title, a coloured status badge, a short description preview, the due date with a calendar icon, and the assigned employee with an avatar. A menu button on the card holds the actions that the current user is allowed to perform. A list/grid toggle changes the layout.

9.2 Filtering, Search and Sorting

- **Status tabs:** All, Pending, In Progress and Completed, each with a live count.
- **Due before:** a date picker that shows only tasks due on or before the selected date.
- **Search:** finds tasks by title or description.
- **Sort:** orders tasks, for example by due date (earliest first).
- **Result count:** a label such as “Showing 8 of 8 tasks”, and a clear empty state when nothing matches.

The status and due-date filters are sent to the backend as query parameters on GET `/api/tasks`, for example:

```
GET /api/tasks?status=Pending
GET /api/tasks?dueBefore=2026-10-15
```

9.3 Updating Task Status

Employees update the status of a task assigned to them using PATCH `/api/tasks/:id/status`. The selector offers only Pending, In Progress and Completed. The ownership middleware checks that the task belongs to the logged-in employee, otherwise the backend returns 403 Forbidden. Managers can also update status.

9.4 Deleting Tasks

Deleting uses a confirmation dialog that names the task and warns that the action cannot be undone, with Cancel and Delete buttons. The delete rules are enforced in the deleteTask controller:

- A manager can delete any task.
- An employee can delete a task only if it is assigned to them and its status is Completed.
- The delete action is hidden in the interface when the user is not allowed to use it, and the backend rejects the request anyway if it is attempted.

10. Create Task (Manager)

Only managers can create tasks. The Create Task page is a structured form split into Task Details and Assignment sections.

Figure 10.1 — Create Task form with live preview and tips panel.

Field	Required	Description
Task Title	Yes	Short, action-oriented summary of the deliverable
Description	Yes	Instructions and acceptance criteria, with a character counter
Assign Employee	Yes	Dropdown filled from GET /api/auth/employees; employees are never hard-coded
Due Date	Yes	Date picker with quick buttons: Today, Tomorrow, In 3 days, Next week

A Live Preview card on the right updates as the manager types and shows how the task will look, including the default Pending status. A Tips panel suggests writing action titles, clear acceptance criteria and realistic dates. The form shows validation messages, a loading state while the task is being created, and success or error feedback. The request is sent to POST /api/tasks, which is restricted to the manager role.

11. Profile

The Profile page summarises the signed-in account.

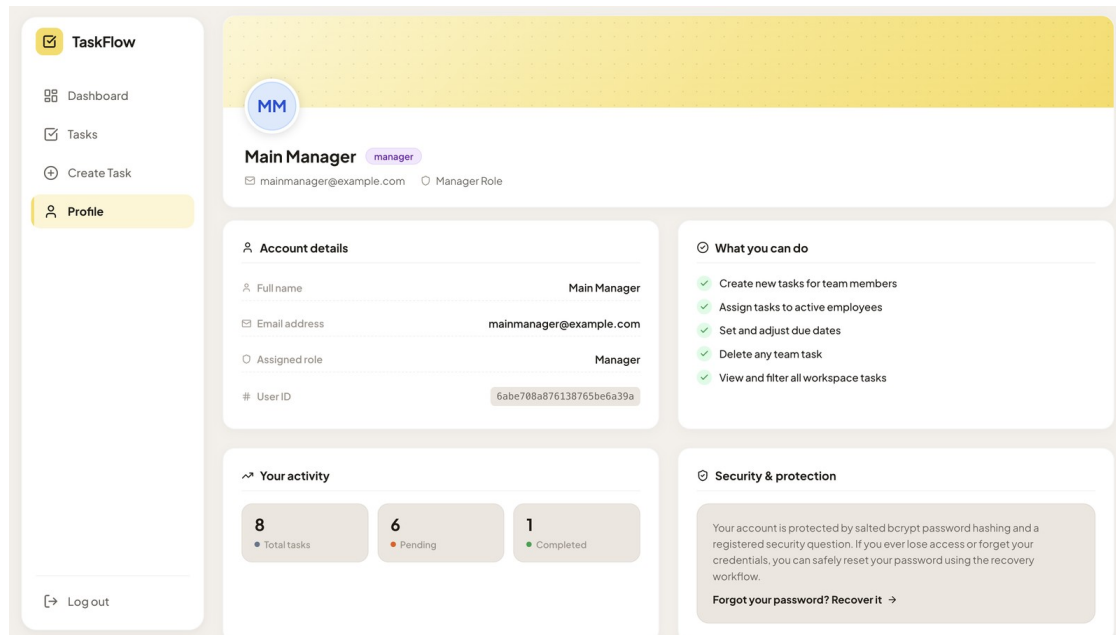


Figure 11.1 — Profile page for the manager account.

- **Header:** avatar with initials, name, role badge and email.
- **Account details:** full name, email address, assigned role and user ID.
- **What you can do:** a checklist of the permissions of the current role (for a manager: create tasks, assign tasks to active employees, set and adjust due dates, delete any team task, and view and filter all workspace tasks).
- **Your activity:** counts of total, pending and completed tasks.
- **Security & protection:** explains that the account uses salted bcrypt hashing and a registered security question, with a shortcut to the password recovery flow.

12. Employee Workflow

Employees use the same application with a reduced set of controls. Their navigation shows Dashboard, Tasks, Update Status and Profile, and the manager-only Create Task page is not shown.

- The Tasks page and dashboard list only the tasks assigned to the employee, because the backend filters the results by the logged-in user.
- The employee can change the status of an assigned task between Pending, In Progress and Completed.
- The delete option appears only on tasks that are assigned to the employee and already Completed.
- Opening or modifying a task that belongs to another employee is rejected with 403 Forbidden.
- Status filters, due-date filter and search work in the same way as for managers.

13. API Reference

All endpoints are served from <http://localhost:5001/api> during development.

13.1 Authentication Endpoints (/api/auth)

Endpoint	Method	Access	Description
/register	POST	Public	Register with security question and answer
/login	POST	Public	Validate credentials and return a JWT
/forgot-password	POST	Public, rate limited	Return the security question for an email
/verify-security-answer	POST	Public, rate limited	Check the answer, apply logout, issue reset token
/reset-password	POST	Public, rate limited	Set a new password using the reset token
/create-manager	POST	Manager	Create a manager account
/employees	GET	Manager	List employees for the assignment dropdown

13.2 Task Endpoints (/api/tasks)

Endpoint	Method	Authorization Rule	Description
/	GET	Authenticated	Manager sees all (filterable); employee sees assigned tasks only
/:id	GET	Authenticated + ownership	Get one task
/	POST	Manager only	Create and assign a task
/:id	PUT	Manager only	Update task details
/:id/status	PATCH	Assigned employee or manager	Update task status
/:id	DELETE	Manager, or assigned employee when Completed	Delete a task

14. Security and Authorization

Security is enforced on the server. The frontend hides controls that a user may not use, but this is only for usability and is never relied on for protection.

Area	How It Is Handled
Authentication	JWT verified by the protect middleware on every private route.
Role-based access	authorize('manager') restricts task creation, task editing and employee listing to managers.
Ownership-based access	checkTaskOwnership compares the task's assigned employee with the logged-in employee before status updates and single-task access.
Delete rule	The deleteTask controller allows managers, or an employee only for their own Completed task.
Password storage	bcryptjs with 10 salt rounds for passwords and security answers.
Brute-force defence	Endpoint-specific rate limits and a 15-minute lockout after 5 wrong security answers.
Secrets	JWT_SECRET, MONGO_URI and PORT are loaded from the .env file and are not committed to Git.
Error responses	Clear status codes (400, 401, 403, 404, 429) without stack traces or database internals.

The frontend converts these status codes into readable toast messages and banners, for example “You are not authorized to perform this action.”

15. Testing and Verification

A final requirements audit of the backend and frontend was carried out on October 1, 2026. It evaluated functional requirements, authentication, authorization, task lifecycle, password recovery and rate limiting, and reported full compliance with the assignment specification. The audit results are summarised below.

15.1 Requirements Compliance

Requirement	Implementation	Result
Separate Employee and Manager roles with JWT	Two Mongoose models, role in JWT payload, role middleware	Passed
Managers assign tasks to employees	POST /api/tasks restricted to managers	Passed
Employees update status of assigned tasks	PATCH /api/tasks/:id/status with ownership check	Passed
Employees cannot modify unassigned tasks	Assignment check returns 403 Forbidden	Passed
Filtering of tasks	Query parameters handled in getTasks	Passed
Delete rules	Manager: any task. Employee: own task and Completed	Passed
Security question recovery	Three-step flow with reset tokens	Passed
Rate limiting and lockout	Limiters on recovery routes, lock after 5 failures	Passed
Hashed passwords and answers	bcryptjs, 10 salt rounds	Passed
No hard-coded secrets	Values read from .env through dotenv	Passed

15.2 Functional Test Scenarios

Role	Scenario	Expected Result
Employee	Log in and open the dashboard and Tasks page	Only assigned tasks are shown
Employee	Apply status and due-date filters	List and count update correctly
Employee	Change status of an assigned task	Status is saved and badge updates
Employee	Delete an own task with status Completed	Task is deleted after confirmation
Employee	Delete an own task with status Pending	Not allowed; delete is hidden and the API rejects it
Employee	Access another employee's task	Rejected with 403 Forbidden
Employee	Run the forgot-password flow	Password is reset and user returns to Login
Manager	Log in, view dashboard and all tasks	All tasks and team workload are visible
Manager	Create a task and assign an employee with a due date	Task appears in the list as Pending
Manager	Delete any task	Task is deleted after confirmation

After every interface change, the frontend is built with `npm run build` from the frontend folder to confirm that the project still compiles.

16. Conclusion

TaskFlow gives a small company a clear and secure way to assign work and track progress. Managers can create tasks, assign them to employees and set due dates, while employees can see their own tasks and update their status. Role-based and ownership-based authorization on the backend ensures that unauthorized changes are rejected.

The project meets the objectives of Project 120. It combines a well-structured Express and MongoDB backend with JWT authentication, security-question password recovery, rate limiting and account lockout, together with a responsive React interface that includes a dashboard, filterable task list, task creation form and profile page.

17. GitHub Repository Link

The complete source code and project files for the TaskFlow Task Management System are available at the following GitHub repository:

Repository: <https://github.com/Yuvrajmishra-cell/Task-Management-System.git>

Repository name: Task-Management-System

The repository contains the backend and frontend source code and the related project materials used to implement and demonstrate the system. The .env file and node_modules folder are excluded through .gitignore.